M4GB persistent server vs one-shot process on the Weil-descent ladder (2026-11-11)

M4GB as a persistent server: one process per binary instead of one per call (2026-11-11)

Owner's question (2026-11-11): "Any other perf optimizations here? Can we use tricks like dynamic programming or things that I'm not aware of for Gröbner bases?" The catalogue of levers, with what this repository already does and what remains, is `docs/ic/GROEBNER_PERF_LEVERS_20261111.md`. This study measures the first lever on that list, the only one that needs no change to the algorithm: the end-to-end ladder (`../m4gb_e2e_ic_ladder_20261110/`) spends one process start, one field-table build and one thread-pool start per decomposition call, and at 12 to 16 unknowns the solve itself takes about as long as that fixed cost.

What changed

- M4GB patch `0003-serve-mode-persistent-solver-process.patch` (`third_party/m4gb-patches/`): a `--serve` mode. One solver instance reads `SOLVE <in> <out>` lines from stdin, runs each through the unchanged `solve()` entry point (which re-initialises the solver's state), writes the output file and answers `OK`, `UNSAT` or `ERR <what>`; `QUIT` or end of input stops it. The one-shot `--solve` path is untouched; upstream's algorithm is untouched.
- Rust adapter `run_m4gb_persistent` in `src/cryptanalysis/m4gb_oracle.rs`: one server per binary (`MAXVARS` ladder), kept in a process-wide pool, one request at a time per server, the same deadline as the one-shot path (the server is killed and replaced on a timeout). Registered as the engine `m4gb-f2-server` beside `m4gb-f2`, so `ic descent --solver` prices both on the same targets in one invocation.

Protocol

`run_ab.sh` under `tools/benchrec.py run`: `ic descent --json --solver inherited-f4 --solver m4gb-f2 --solver m4gb-f2-server`, one invocation per cell and family, 6 seeded targets, 3 repeats, 60 s budget per call, one thread (`M4GB_THREADS=1`), binaries `solver_m4gb_n{16,20,24,28,32}_gf2` from

`scripts/m4gb/build.sh` at the pinned upstream commit with patches 0001 to 0003. Cells: Koblitz (K) and random binary (R), m = 2 at n = 11 to 23 (12 to 24 unknowns), m = 3 at n = 11, 13, 15 (12 to 15 unknowns). The runs are interleaved per target and the engine order rotates per repetition, so host drift falls on every engine. Local container, no isolation; a stage diagnostic, not a leaderboard row.

Raw mechanism check before the run (100 calls on one 16-variable descent system, `research/m4gb_integration_20261110`'s `wd_n16.in`, shell timing, two rounds) `[pcc:unverified]`: one-shot 30.7 and 31.3 ms per call, server 24.3 and 26.7 ms per call, process start plus option parsing alone 4.1 ms per call; the two output files are byte-identical.

Pre-stated expectation and falsifier

Expectation: the server removes a constant 5 to 15 ms per call (process start, field tables, thread pool, and on the Rust side the pipe set-up), so the server-to-one-shot ratio is lowest at 12 unknowns and rises towards 1 as the solve grows past a few hundred milliseconds; the decisions agree on every target. Falsifier: a ratio at or above 1 on a cell where the one-shot call takes under 50 ms, or any disagreement with the reference, means the server path costs more than it saves (pipe round trip, file writes, lock) or is wrong.

Results

Filled after the run (`RESULTS.md`, `results.json`, `results.svg/png`, `REPORT.pdf`).

Median over targets of the per-target median wall, ms, one call. Ratio columns: `m4gb-f2-server` over `m4gb-f2` (the persistent server over the one-shot process), the median over targets of the paired per-target saving in ms with the count of targets on which the median wall of the server is below that of the one-shot process, and the server over the reference engine of the cell. `dec` = targets decided in every repetition / targets; `ob` = targets with a budget verdict; `agree` = every decided answer equals the reference.

family	n	n'	m	vars	ref	inherited-f4 ms (dec, ob, agree, Dmax)	m4gb-f2 ms (dec, ob, agree, Dmax)	m4gb-f2-server ms (dec, ob, agree, Dmax)	serve r one sho
K	11	6	2	12	inherited-f4	0.53 (6/6, 0, y, 3)	8.90 (6/6, 0, y, 3)	3.22 (6/6, 0, y, -)	0.36›
R	11	6	2	12	inherited-f4	0.65 (6/6, 0, y, 3)	9.00 (6/6, 0, y, 3)	4.12 (6/6, 0, y, -)	0.46›
K	13	7	2	14	inherited-f4	0.87 (6/6, 0, y, 3)	15.0 (6/6, 0, y, 3)	7.52 (6/6, 0, y, -)	0.50›

family	n	n’	m	vars	ref	inherited-f4 ms (dec, ob, agree, Dmax)	m4gb-f2 ms (dec, ob, agree, Dmax)	m4gb-f2-server ms (dec, ob, agree, Dmax)	server one- sho
R	13	7	2	14	inherited-f4	1.20 (6/6, 0, y, 3)	15.4 (6/6, 0, y, 3)	10.9 (6/6, 0, y, -)	0.71›
K	15	8	2	16	inherited-f4	1.98 (6/6, 0, y, 3)	20.1 (6/6, 0, y, 4)	13.9 (6/6, 0, y, -)	0.69›
R	15	8	2	16	inherited-f4	1.71 (6/6, 0, y, 3)	22.7 (6/6, 0, y, 3)	17.0 (6/6, 0, y, -)	0.75›
K	17	9	2	18	inherited-f4	2.54 (6/6, 0, y, 3)	35.3 (6/6, 0, y, 3)	35.3 (6/6, 0, y, -)	1.00›
R	17	9	2	18	inherited-f4	2.42 (6/6, 0, y, 3)	27.9 (6/6, 0, y, 3)	23.9 (6/6, 0, y, -)	0.85›
K	19	10	2	20	inherited-f4	4.20 (6/6, 0, y, 3)	33.3 (6/6, 0, y, 3)	26.5 (6/6, 0, y, -)	0.80›
R	19	10	2	20	inherited-f4	5.42 (6/6, 0, y, 3)	45.3 (6/6, 0, y, 3)	37.2 (6/6, 0, y, -)	0.82›
R	21	11	2	22	inherited-f4	7.92 (6/6, 0, y, 3)	196.3 (6/6, 0, y, 4)	170.2 (6/6, 0, y, -)	0.87›
K	23	12	2	24	inherited-f4	14.5 (6/6, 0, y, 3)	421.2 (6/6, 0, y, 4)	402.0 (6/6, 0, y, -)	0.95›
R	23	12	2	24	inherited-f4	14.3 (6/6, 0, y, 3)	289.4 (6/6, 0, y, 4)	285.7 (6/6, 0, y, -)	0.99›
K	11	4	3	12	inherited-f4	22.2 (6/6, 0, y, 6)	6206.8 (6/6, 0, y, 7)	6364.0 (6/6, 0, y, -)	1.03›
R	11	4	3	12	inherited-f4	24.0 (6/6, 0, y, 6)	6123.5 (6/6, 0, y, 7)	6457.5 (6/6, 0, y, -)	1.05›
K	13	5	3	15	inherited-f4	186.4 (6/6, 0, y, 6)	46186.5 (2/6, 4, y, 7)	45441.3 (2/6, 4, y, -)	0.98›
R	13	5	3	15	inherited-f4	176.4 (6/6, 0, y, 6)	46591.1 (2/6, 4,	46297.3 (2/6, 4,	0.99›

family	n	n’	m	vars	ref	inherited-f4 ms (dec, ob, agree, Dmax)	m4gb-f2 ms (dec, ob, agree, Dmax)	m4gb-f2-server ms (dec, ob, agree, Dmax)	serve r one- sho
							y, 7)	y, -)	
K	15	5	3	15	inherited-f4	172.0 (6/6, 0, y, 6)	n/a (0/6, 6, y, -)	n/a (0/6, 6, y, -)	n/a
R	15	5	3	15	inherited-f4	174.8 (6/6, 0, y, 6)	n/a (0/6, 6, y, -)	n/a (0/6, 6, y, -)	n/a